

ZLC FPGA Pulse Streamer ????

address-switch entryptpoint / GUI subset compile / server contract

Vivado 2019.x runtime edge table?Pulse GUI ? sequencer server ??

2026-06-05

Contents

1	结论先行	1
2	入口和目录	3
3	编译契约	4
4	scan 和容量	6
5	analog bus segment 验证	7
6	验证摘要	9
7	上机顺序	10

1

结论先行

本报告记录当前仓库中 FPGA pulse-streamer 的 address-switch 入口、Pulse GUI/API 编译边界、server 启动边界和文档同步状态。当前硬件路线是固定 address-switch runtime pulse-streamer: GUI 可以只显示或配置少数 channel, 但 server 上传给 FPGA 的 program 始终按 XDC 推断的完整 channel order 补齐, 未使用 channel 的 mask bit 为 0。

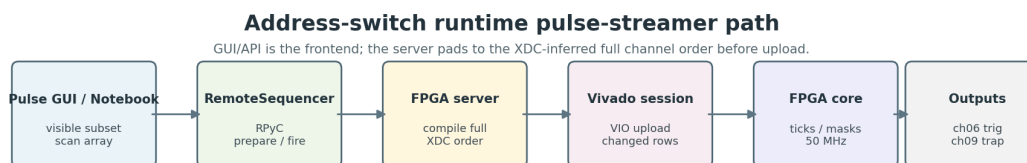


Figure 1.1: 控制电脑、FPGA server、Vivado/VIO、FPGA bitstream 和相机外触发的边界。GUI 在前端; full-width 补齐发生在 sequencer compile/upload 边界。

本轮重点检查:

- `fpga\build _and_program.bat` 是 build/program/check/diagnose 的统一入口。
- `fpga\run _server.bat` 从 address-switch XDC 推断完整 channel list, 默认 fallback 为 `ch00 ... ch61`。
- trigger channel 从 XDC label `emCCD` 推断, 默认是 `ch11`。
- `pulses\camera _imaging_address_switch.json` 默认只显示 `ch09/ch00/ch03/ch11`, 但保存和编译仍保留完整硬件顺序。
- GUI/API state 即使只包含相机成像子集, 也会编译到 full-width program; 未配置 channel 全

部为 off。

- `da_dipole/da_bias_y/da_bias_x/da_bias_z` 不再通过普通 TTL mask 展开 ramp; 它们使用独立 analog bus segment table。

2

入口和目录

从 repo 根目录运行:

Command line

```
.\fpga\build_and_program.bat --help
.\fpga\build_and_program.bat --check
.\fpga\build_and_program.bat
.\fpga\run_server.bat --check-config
.\fpga\run_server.bat
```

默认 Vivado 工程:

Address-switch build

```
fpga\build\address_switch\address_switch.xpr
fpga\build\address_switch\address_switch.runs\impl_1\zlc_pulse_streamer_tj
↪ op_address_switch.bit
fpga\build\address_switch\address_switch.runs\impl_1\zlc_pulse_streamer_tj
↪ op_address_switch.ltx
```

默认 XDC:

XDC

```
references\source_archives\address_switch\address_switch.srcs\constrs_1\nj
↪ ew\addre.xdc
```

若使用另一份已确认的板级 XDC, 设置 [ZLC_PS_XDC](#)。不再需要复制旧 template XDC。

3

编译契约

Pulse GUI 和 notebook 使用 `PulseTableState`。GUI 的 `visible_channels` 只影响显示，不影响 FPGA bit order。上传前，sequencer service 会把 state 对齐到 server 的 full channel list:

Python

```
state =  
↪ na.PulseTableState.load("pulses/camera_imaging_address_switch.json")  
program = state.compile(clock_hz=50_000_000, trigger_channels=["ch11"])
```

默认 camera-imaging preset 的关键通道是:

Camera subset

```
ch09 trap  
ch00 cooling  
ch03 probe  
ch11 emCCD
```

默认 `camera_exposure_ns=19980000` 时, 50 MHz 编译结果为:

camera imaging address-switch

```
ticks: 0, 100000, 105000, 106000, 1105000, 1106000  
masks: 513, 512, 2568, 520, 512, 0  
trigger_count: 1  
repeat_forever: true
```

`513 = ch00 + ch09`, `2568 = ch03 + ch09 + ch11`。这说明相机外触发 bit 是 `ch11/emCCD`, 不是 `ch03`。

camera_imaging_address_switch edge table

edge	state	tick @ 50 MHz	mask
0	load	0	513
1	pre	100000	512
2	trigger	105000	584
3	exposure	106000	520
4	post	1105000	512
5	idle	1106000	0

Figure 3.1: camera_imaging_address_switch preset 的 edge table。图中只使用相机成像子集，上传 program 仍然带完整 channel list。

4

scan 和容量

scan 使用一个 named scan table file, 并由 host 打包到当前两轴 timing scan RAM。单个 FPGA program 使用有限 scan rows; 大 scan file 可以由 API 分 chunk 连续上传运行:

Scan table

```
# vars: camera_exposure_ns(ns), trig_delay(ns)
1000 0
2000 20
4000 40
```

所有 timing 值单位是 ns, 默认 50 MHz 硬件下必须对齐到 20 ns。FPGA 上传的是 edge template 加 packed scan RAM, 不会把 scan row 展开成大量 period column。

build/check 使用资源目标参数作为 sizing guide:

Command line

```
$env:ZLC_PS_RESOURCE_TARGET_PCT = "70"
$env:ZLC_PS_MAX_EDGES = "512"
$env:ZLC_PS_MAX_SCAN_POINTS = "256"
.\fpga\build_and_program.bat --check
```

Vivado `report_utilization` 和 `report_timing_summary` 是最终证据; Python/Tcl 的容量打印是 build 前的预算提示。

5

analog bus segment 验证

address-switch bitstream 仍然保留 62-bit digital `prog_mask`, 但 DA 类 10-bit bus 现在走独立 segment upload。普通 TTL 和 DA bus 是两张表:

Ordinary edge table vs analog bus segment table

The two tables are uploaded together but executed by different runtime logic.

Digital TTL edge table

tick	mask[61:0]
0	trap=1, cooling=1, ...
100	probe=1, emCCD=1, ...
22000000	all off

Good for sparse TTL changes.
Every row rewrites the full mask.

Analog/DA bus segment table

bus	start	stop	v0	v1	mode
0	0	0	350	350	edge
0	100	50000	350	720	ramp
2	0	0	512	512	edge

Good for DA values and ramps.
Ramp shape is generated on FPGA.

Figure 5.1: 普通 edge table 与 analog bus segment table 的执行角色不同。

Bus segment row

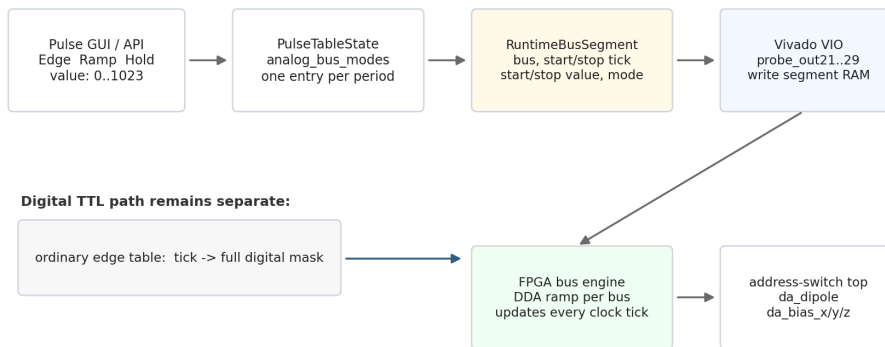
`bus_id, start_tick, stop_tick, start_value, stop_value, mode`

`edge` segment 的 `start_tick = stop_tick`, 运行到该 tick 时直接更新 bus value。 `ramp` segment 使用 `start_tick/stop_tick` 和 `start_value/stop_value`, 由 FPGA 内部 DDA accumulator 在每个 clock tick 计算当前 stair-step。 这样一个长 ramp 只占一个 bus segment, 不会把普通 `tick/mask` edge table 填满。

默认 VIO output probe 从 21 个扩展到 31 个:

Analog/DA bus segment path

GUI/API edits logical 10-bit buses; FPGA stores a compact segment table and generates ramps locally.



Key point: one long DA ramp consumes one bus segment, not hundreds of ordinary TTL edge rows.

Figure 5.2: 从 GUI/API 到 Verilog runtime 的 bus segment 上传和执行路径。

Bus VIO probes

probe_out21	zlc_bus_prog_we	width 1
probe_out22	zlc_bus_prog_bus	width 2
probe_out23	zlc_bus_prog_addr	width 6
probe_out24	zlc_bus_prog_start_tick	width 32
probe_out25	zlc_bus_prog_stop_tick	width 32
probe_out26	zlc_bus_prog_start_value	width 10
probe_out27	zlc_bus_prog_stop_value	width 10
probe_out28	zlc_bus_prog_mode	width 2
probe_out29	zlc_bus_counts	width 28
probe_out30	zlc_scan_prog_bus_values	width 40

四个 bus 的硬件顺序为 `da_dipole`, `da_bias_y`, `da_bias_x`, `da_bias_z`, 每个 bus 最多 64 个 segment。没有使用的 bus 在 `bus_counts` 中 count 为 0, 因此旧 RAM 内容不会被执行。普通 TTL 输出继续使用 `tick/mask` edge rows; 一个长 DA ramp 只消耗一个 bus segment, 不消耗几百个 ordinary edge rows。静态 edge-mode DA value scan 通过 `zlc_scan_prog_bus_values` 每个 scan row 上传一个 packed 4x10-bit 值; 当前限制是 hardware scan array 不能和 analog bus ramp segment 同一次上传, 如果需要扫描 analog ramp, 应按 scan point 逐个 prepare/fire。

6

验证摘要

当前交付需要覆盖的最小检查:

- `python -m py_compile` 覆盖 sequencer、server、frontend GUI 和 fpga backend。
- `python -m json.tool` 覆盖三份 tutorial notebook。
- `fpga\build_and_program.bat -help` 和 `fpga\run_server.bat -help` 显示 address-switch 默认路径、50 MHz clock 和 XDC-inferred channel list。
- `pulses\camera_imaging_address_switch.json` 可编译出上述 ticks/masks。

7

上机顺序

1. 在 Verilog/FPGA 电脑把 repo 放到短路径, 例如 D:\ZLC 。
2. 运行 `install_requirements.bat`。
3. 设置 `ZLC_PS_VIVADO_BIN`, 或确认 Vivado 自动发现路径正确。
4. 运行 `fpga\build _and_program.bat -check`。
5. 运行 `fpga\build _and_program.bat`, 确认 `.bit/.ltx` 生成并 program 成功。
6. 运行 `fpga\run _server.bat -check-config`, 确认 project、bit、ltx、XDC、channel、trigger、clock 都正确。
7. 运行 `fpga\run _server.bat` 保持 server 窗口打开。
8. 在 FPGA 电脑或控制电脑运行 `pulse_gui.bat -remote-host ... -state .\pulses\camera _imaging_address_switch.json`。
9. 按 On Pulse 前确认示波器接到 ch09/ch00/ch03/ch11 对应的物理输出。